

Functions, Part I

CMSC 104 Section 2
February 23, 2026

Administrative Notes

- Classwork 3 is due tonight before midnight
- Project 1 is due Wednesday night (2/25) before midnight

Functions

- A way to jump to another part of the program; execute code there; and then return to wherever you were before you jumped

What is a function?

In math, it's a unique mapping between each input and one output

$$f(x) = x^2 + 2x + 1$$

Put in a value for x , you'll get back one output.

In python, it's sort of the same idea, but not exactly

A block of code, called by its name, that optionally takes input and returns a specific output

An example of a function in Python

This is the name of the function.
It must be a legal Python variable name

The word “def”
means you’re
defining a
function

```
def calculate_days (years, months, days):
```

These are the
parameters.

```
    num_days = 365 * years + 30 * months + days
```

```
    if years > 4:
```

```
        num_days += 1
```

```
    return num_days
```

This code is the
body of the
function

The “return” statement is a
way of passing a value
back to where the function
was called from

Calling a function

To call the function from the previous slide, `calculate_days`:

```
yrs = 23
```

```
mos = 3
```

```
days = 22
```

```
length_of_time = calculate_days(yrs, mos, days)
```

These are arguments. They will be matched up with parameters, in order

If the function returns a value, you have to store it somewhere if you want to use it

Call the function by using its name, with appropriate arguments

Why use functions?

Use a function whenever your program is going to do the same thing multiple times

- No, not like in a loop. In different *parts* of the program.
- You **could** rewrite the code each time, but you're likely to do it differently somewhere
- You could just copy and paste the same code each time, but what if you're just copying and pasting a bug?

Using functions simplifies the program and makes it easier to get the program correct

Functions vs. methods

`calculate_days()` is a *function*; `str.split()` is a *method*. What's the difference?

A ***method*** is explicitly tied to ***one object***; the object on which it is invoked.

A ***function*** is not tied to anything; it operates on the values passed to its parameters (if there are any).

Hopefully this becomes clear over the next couple of days!

Built-in functions and user-defined functions

Python has a number of built-in functions that you've already used:

```
print("Hello, world") # print is a function; "Hello, world" is its argument.
```

```
len(num_list) # len is a function; num_list is its argument. It returns  
the number of elements in num_list
```

You can add to the built_in functions with functions that you will define yourself.
Like `calculate_days`, a few slides ago.

```
My_list = [4,3,2,1,0]
```

```
X = len(My_list) # function call; returns an int; storing it in X
```

Where do you define functions?

Usually, above the main program - but that's **NOT** required.

Start with your header
comment

```
LABELS = ['A', 'B', 'Others']
```

Now define your
constants

```
def calculate_days(days, months, years)
```

Then define your
functions

```
If __name__ == "__main__":
```

And finally your main
program

Where do you call functions?

Anywhere other than on the left hand side of an equals sign

The call to a function can be anywhere in the program where the function is needed

Can a function call another function?

- YES - generally called a “helper function”

Can a function call itself?

- YES; that's called 'recursion' and we'll get to that

Can a function call the main program?

- NO; it can return values to the main program but that's all

Does a function have to be defined before it can be called?

NO!!!! You can call a function before it's defined!!! Python can handle this.

If Python sees something that looks like a function call, it will simply wait for that function to be defined.

Matching arguments and parameters

I said that parameters and arguments are matched in order. What does that mean?

```
def subtract(x, y):  
    print(x, "-", y, "=", str(x-y))  
if __name__ == "__main__":  
    x = 3  
    y = 4  
    subtract(y, x)
```

What happens if the arguments and parameters don't match?

```
def subtract(x, y):  
    print(x, "-", y, "=", str(x-y))  
if __name__ == "__main__":  
    x = 3  
    y = 4  
    subtract(y)
```

The return statement

return is the statement used to pass values from the function back to the main program (or back to the calling function)

The syntax is

return variable_name # you can optionally put the variable name in ()

For this class, a function will return one value.

Using returned values

If you want to use the value that's returned, you have to do so in the calling program/function

If the function contains:

```
def factorial(num):  
    Product = 1  
    For i in range(num):  
        Product *= i  
    return product
```

The calling program should use the value:

```
fact = factorial(5)
```

Or

```
sum = factorial(7) + factorial(2)
```

or...

Example - calculate the “fourth root” of a positive number

```
# the “fourth root” of a number is that number raised to the ( $\frac{1}{4}$ ) power
```

```
#
```

```
def fourth_root(num):
```

```
    ans = num**(1/4)
```

```
    return (ans)
```

```
#set the variable's value, then call the function
```

```
original_number = 81
```

```
final_number = fourth_root(original_number)
```

```
print(final_number)
```

Another example: reverse_word

```
# First the function definition
def reverse_word (word)
# now the code. DON'T FORGET TO INDENT!!
    i = 0
    reversed_word = '' #Do not use the function name here!!!!
    while i < len(word):
        reversed_word = reversed_word + word [i]
    print (" The word ", word, " reversed is ", reversed_word)
```

This function does its work independently; it doesn't return any values for use in the program. It would be more useful if this function did return a value. Let's revise it.

Revised reverse_word

First the function definition

```
def reverse_word (word):  
    # now the code. DON'T FORGET TO INDENT!!  
  
    i = 0  
    reversed_word = '' #Do not use the function name  
    here!!!!  
    while i < len(word):  
        reversed_word = reversed_word + word[-(i+1)]  
        i += 1  
    print (' The word ', word, ' reversed is ',  
reversed_word)  
  
...
```

Now the call is

```
animals = ["cat", "Australian cattle dog",  
"duckbilled platypus", "ocelot", "zebra"]  
for critter in animals:
```

```
    r_word = reverse_word(critter)
```

```
# now do whatever you want to with the
```

```
# reversed word in the main program
```

Program control in a function

A function stops executing once it executes a `return ()` statement. Let's look at `fourth_root()` again. This time we'll add some code after the “`return()`” statement

the “fourth root” is the number raised to the $\frac{1}{4}$ power

```
def fourth_root(num):
```

```
    ans = num**(1/4)
```

```
    return (ans)
```

The following statement is not going to execute, because the function has already ended

due to the “return” statement

```
    print(“the code has successfully executed”)
```

Using the returned value from a function

The calling function gets the returned value as if the call were a variable.

Consider the built in function `len(some_list)`:

```
if len(some_list) > 2:

    print(len(some_list))

    list_length = len(some_list)
```

The only place you can't use the returned value is on the left side of an assignment statement

```
len(some_list) = x + 5 # NOT PERMITTED
```

None and NoneType

The original reverse_word function was:

```
def reverse_word (word):  
    # now the code. DON'T FORGET TO INDENT!!  
  
    i = 0  
    reversed_word = '' #Do not use the function name here!!!!  
    while i < len(word):  
        reversed_word = reversed_word + word[-(i+1)]  
        i += 1  
    print (' The word ', word, ' reversed is ', reversed_word)
```

There's no “return()” statement, so what does this return?

A special value called “None” which is of type “NoneType”

The value “None”

Python defines a special value “None” which is of type “NoneType”

If a function does not otherwise return a value, it returns “None”

- Functions do not have to contain a return statement. Any function without a return statement returns “None.”
- If a function’s return statement is not executed, the function returns “None.”
- You can explicitly tell the function to return “None.”

A common error is to have a function return None when you expected it to return something else.

- You’ll see an error message like:
- `Error; type 'NoneType' is not iterable`

“None”

If a function returns “None” you will generally have trouble using that value in your code, unless you use it for error checking. Note: not checking whether your function returned “None” is a very common error, and can make debugging difficult. Check for that in your calling code.

In the code in your function, having

```
    return None
```

```
    return
```

And no return statement at all have the same effect - your function returns a value of None.

Error checking using “None”

Function definition

```
def fourth_root(num):  
    if num >= 0:  
        ans = num**(1 / 4)  
        return (ans)  
    else:  
        return None
```

we could have also said

```
#         return
```

or just omitted the entire else: clause and

have no return statement at all. The code

works the same

get the original value from the user

then call the function

```
original_number = float(input("enter a number"))
```

```
done = False
```

```
while not(done):
```

```
    final_number = fourth_root(original_number)
```

```
    if final_number != None:
```

```
        print("The fourth root of", original_number, end = " ")
```

```
        print(" is ", final_number)
```

```
        done = True
```

```
    else:
```

```
        original_number = float(input("we were serious about needing a  
non-negative number"))
```

More on calling by value

First the function definition

```
def reverse_word (word):
```

now the code. DON'T FORGET TO INDENT!!

```
    i = 0
```

```
    reversed_word = ''
```

```
    while i < len(word):
```

```
        reversed_word = reversed_word + word
```

```
    [-(i+1)]
```

```
#     return(reversed_word)
```

```
...     word = reversed_word
```

Now the call is

```
animals = ["cat", "Australian cattle dog",  
           "duckbilled platypus", "ocelot", "zebra"]
```

```
for critter in animals:
```

```
    reverse_word(critter)
```

```
    print(critter)
```

now do whatever you want to with the

reversed word in the main program

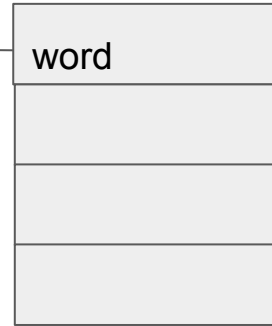
This doesn't work in Python!! (It does work in some other languages, so if you've got experience with doing this, put it out of your mind for now)

Why doesn't this work?

These are NOT the same locations in memory. The value is copied over when the function call is made. Nothing is copied back to the main program, except what's in "return"



Memory for
main program



Memory for
reverse_word

Scope of a variable

Scope of variables

Scope means where a variable can be directly seen and used

Variables defined in the main program can be seen everywhere in the program. BUT - you should only directly use them in the main program. If you need their values in a function, pass them as arguments. If you use a main program variable directly in a function, without passing it as an argument, that's called a "global variable" and it will cost you major points!!!

Variables defined in functions can only be seen and used in the functions where they are defined. These are called "local variables."

Examples of variable scope

Calculating someone's age in days - slight modification of Monday's code

Pseudocode - program computes someone's age in days.

- Ask the user for today's date
- Ask the user for user's birthdate
- Calculate the year and day_of_year for today, and for the user's birthdate
 - 3/10/2021 69th day of 2021
 - Born: 4/15/1989 105th day of 1989 subtract 4/15/1989 from 3/10/2021 - 3 subtractions; 2 carries - complicated; easy to get wrong. 105 1989 subtracted from 69 2021 easier to get right (2 subtractions, one carry)
- Subtract; print the answer

Importing modules and functions in Python

Python comes with some “builtin” functions such as `len()`, `print()`, `input()`,...

There are tons of other functions that have already been written by others, and which are free to you to use in your programming career.

- There's no need to rewrite a function if you know somebody else has already written it

You get access to that code by using the `import()` function

import()

import() tells the Python interpreter that you want access to a module that you know about, and the functions in that module

A “module” is Pythonic for a group of functions made available. Other languages might call this a “library” or a “package.”

Import random

Imports a module that contains a bunch of functions all related to the generation and management of “random” numbers

Note: the module must be present on your computer for “import” to work. If you get an error message saying the module does not exist, you’ll have to install it.

Using a function in a module

Once you have imported a module, you can use its functions in your program

```
random.randint(1,25)
```

Generates a random integer between 1 and 25, inclusive

You can use this just like any other function:

```
for i in range(10):  
    r_num = random.randint(1,25)  
    print(r_num)
```

How do you know what functions are in a module?

...and what parameters to use to call them?

This is where the ability to search the web is your friend. :-)

All the common modules are documented out there in Python-land, along with their Application Programming Interfaces (APIs)

- Which is a fancy way of saying “descriptions of how to call a function, what the parameters are, what the parameters mean and what the return values are.”